

Checklist

Cost, Performance, and Scalability

1. Define goals and trade-offs

- Define availability and performance SLOs.
- Identify acceptable latency, throughput, and downtime.
- Calculate the cost required to meet each SLO.
- Make cost/performance/scalability trade-offs explicit.
- Confirm that infrastructure cost is justified by the business value delivered.
- Avoid choosing infrastructure based only on the lowest price.

2. Establish cost observability

- Enable cloud-native cost reporting.
- Install or configure OpenCost for Kubernetes cost visibility.
- Create consistent Kubernetes labels for:
 - Team
 - Cost center
 - Business unit
 - Application
- Apply cost labels consistently to namespaces and deployments.
- Track CPU cost.
- Track memory allocation and memory cost.
- Build Grafana dashboards for cost reporting.
- Allocate workload costs to specific teams or projects.
- Establish showback or chargeback reporting.

3. Establish a workload baseline

- Select the application to optimize.
- Record its current hourly and monthly cost.
- Set a measurable cost-reduction target.
- Monitor actual CPU usage.
- Monitor actual memory usage.
- Identify peak CPU periods.
- Identify peak memory periods.
- Determine whether CPU and memory peaks occur at the same time.
- Compare actual usage with configured resource requests.

4. Rightsize workloads

- Reduce oversized CPU requests while leaving reasonable headroom.
- Reduce oversized memory requests while leaving reasonable headroom.
- Determine whether the workload is:
 - CPU-bound
 - Memory-bound
 - I/O-bound
 - Storage-bound
- Select an appropriate instance category for the workload.
- Consider compute-optimized instances for CPU-heavy workloads.
- Consider memory-optimized instances for memory-heavy workloads.
- Consider automated instance selection with Karpenter.

5. Configure Horizontal Pod Autoscaling

- Configure HPA for workloads that benefit from additional replicas.
- Set appropriate minimum replicas.
- Set appropriate maximum replicas.
- Configure CPU utilization targets.
- Configure memory utilization targets where appropriate.
- Configure safe scale-up behavior.
- Configure safe scale-down behavior.
- Add stabilization windows to reduce scaling thrash.
- Consider custom business-level metrics such as:
 - Requests per second
 - Queue depth
 - Active sessions
- Test HPA under generated load.
- Verify that scaling maintains the required SLO.

6. Evaluate Vertical Pod Autoscaling

- Install VPA if required.
- Run VPA in recommendation/"off" mode first.
- Allow VPA enough time to observe realistic workload behavior.
- Review CPU recommendations.
- Review memory recommendations.
- Use Goldilocks if desired to inspect recommendations.
- Implement high-confidence recommendations.
- Define minimum and maximum resource limits.
- Exclude containers such as sidecars when they should not be modified.
- Determine whether HPA and VPA should be used together.

7. Optimize infrastructure capacity

- Identify stable baseline capacity.
- Use on-demand capacity where stability is required.
- Evaluate spot instances for fault-tolerant workloads.
- Avoid spot instances for unsuitable workloads such as:
 - Business-critical single replicas
 - Long-running jobs that cannot tolerate interruption
 - Highly latency-sensitive operations
- Configure interruption handling for spot workloads.
- Configure Kubernetes taints/tolerations where spot nodes require workload isolation.
- Configure node affinity to prefer spot capacity where appropriate.
- Allow fallback to on-demand capacity when needed.
- Evaluate committed-use discounts for predictable baseline workloads.
- Avoid overcommitting beyond the true baseline.
- Avoid using committed capacity for highly variable development, test, batch, or emerging workloads.

8. Implement cost governance

- Define ResourceQuotas for each team or namespace.
- Set CPU request quotas.
- Set memory request quotas.
- Set CPU limits.
- Set memory limits.

- Limit maximum pod counts where appropriate.
- Limit persistent volume claims where appropriate.
- Configure LimitRanges for individual containers.
- Define default resource requests.
- Define default resource limits.
- Define minimum and maximum allowable resources.
- Require workloads to declare CPU requests.
- Require workloads to declare memory requests.
- Enforce resource requirements with policy-as-code tools such as Kyverno or Gatekeeper.

9. Configure cost anomaly detection

- Establish a normal cost baseline.
- Alert on unexpected cost spikes.
- Compare current spending rates with historical averages.
- Tune anomaly thresholds based on observed production behavior.
- Investigate memory leaks that continuously increase resource consumption.
- Investigate workloads consuming unexpectedly high CPU.
- Review and refine alerts over time.

10. Integrate cost controls into CI/CD

- Estimate workload cost before production deployment.
- Compare new CPU requests with the previous deployment.
- Compare new memory requests with the previous deployment.
- Flag unusually large increases in resource requirements.
- Require explicit approval or override for major cost increases.
- Check namespace ResourceQuotas during deployment.
- Fail deployments that would exceed namespace budgets.
- Provide a clear remediation message when a quota check fails.
- Add estimated monthly cost to deployment metadata.
- Add cost information to pull-request or deployment summaries.
- Compare post-deployment cost with the previous seven-day average.

11. Measure optimization results

- Compare post-optimization cost with the original baseline.
- Verify that performance has not degraded.

- Verify that availability SLOs are still being met.
- Verify that latency remains acceptable under load.
- Verify that autoscaling responds correctly.
- Identify any remaining waste.
- Repeat optimization where necessary.

12. Exercise 12.3 Completion Checklist

- Establish the application's baseline cost.
- Set a **30% cost-reduction target**.
- Profile CPU and memory usage for **1 week**.
- Rightsize resource requests using observed usage.
- Configure HPA targeting approximately **75% CPU utilization**.
- Load-test the HPA configuration.
- Run VPA in recommendation/"off" mode.
- Review VPA/Goldilocks recommendations after **1 week**.
- Evaluate whether a different instance type would be cheaper.
- Measure results after approximately **2 weeks**.
- Determine whether the 30% reduction target was achieved.
- Document the changes made.
- Document the cost and performance results.
- Share the findings with the platform team.

Final review

- Cost is visible and attributable.
- SLOs are defined and measurable.
- Workloads are appropriately sized.
- Autoscaling is configured where beneficial.
- Capacity uses the right mix of on-demand, spot, and committed resources.
- Cost guardrails prevent uncontrolled consumption.
- Cost anomalies trigger alerts.
- CI/CD exposes the financial impact of deployments.
- Optimization is treated as a continuous platform practice rather than a one-time project.